LAB 6
Code and Branch Coverage Analysis

Pizza Order Application

Arundhathi K | PES1UG24CS082 | Section 5B
CSE, PES University, Bengaluru
3 November 2026

Measurement	Achieved	Required
Code / statement coverage	34 / 34 = 100%	At least 90%
Branch-only coverage	24 / 24 = 100%	At least 90%
Code coverage tests	14 passed	Separate Python file
Branch coverage tests	31 passed	Separate Python file

Objective and scope

Measure the executable statements and decision outcomes in the assigned pizza_order.py module, then improve the starter tests to meet the mandatory 90% thresholds. The original application and starter tests are retained unchanged. Only pizza_order.py is included in coverage measurement.

Assigned repository: SETAPESU26/L6_L7_24_pizza_order
Source commit: 87fde212d6e8c609a17f2a0404c05ad5a113befb
Submission repository: ArundhathiK29/SE-Labs-PES1UG24CS082 / Lab6

Evidence status: Coverage runs and logs are verified. Native terminal screenshots are still required: automated desktop capture was unavailable. The output panels in this report reproduce actual logs and are not presented as screenshots. Add the two native screenshots before final submission.

1. Baseline and final analysis

Test suite	Passed	Statement coverage	Branch-only coverage
Original starter	2	14 / 34 = 41.18%	6 / 24 = 25.00%
Code coverage	14	34 / 34 = 100%	Not measured
Branch coverage	31	34 / 34 = 100%	24 / 24 = 100%

The starter tests exercise a plain medium pizza and an on-time delivery. They omit invalid orders, discounted toppings, cheese burst, all order discounts, long-distance rejection and late delivery outcomes. Twenty executable statements and eighteen branches were initially untested.

Statement coverage is executed statements divided by executable statements. **Branch coverage** is executed decision outcomes divided by possible outcomes. Executing each line alone does not establish that both outcomes of every decision have been tested.

coverage.py displays a combined statement-and-branch percentage in the Cover column when --branch is enabled. The baseline combined value is (14 + 6) / (34 + 24) = 34.48%; its branch-only value is 6 / 24 = 25%. The final branch run has 100% for both measures. JSON counts and check_branch_coverage.py verify the branch-only percentage independently.

2. Decision and boundary coverage

Function	Cases and outcomes exercised
pizza_price	All sizes; invalid size; negative and zero toppings; 3 / 4 topping threshold; multiple reduced-price toppings; cheese burst enabled / disabled; small-size cheese-burst rejection.
order_discount	Totals 299 / 300; Tuesday; both weekend days; ordinary weekdays; student / non-student; integer floor division at 301, 309 and 310.
delivery_status	30 / 31 and 45 / 46 minute boundaries; distance exactly 10 and above 10; far-distance rejection takes precedence over time.

Both final suites assert independent expected values. Exception tests check the ValueError message as well as its type. Full coverage establishes that the measured statements and branches ran; it does not prove correctness for every possible input or establish complete path coverage. Inputs outside the assigned implementation contract are not added as new requirements.

3. Code coverage evidence - actual command output

The following is the unedited command log, wrapped only for page width. It is a text reproduction, not a native terminal screenshot. The source module contains 34 executable statements, all covered by the 14 tests.

```
LAB 6 | PES1UG24CS082 | CODE COVERAGE
Target: pizza_order.py

> python -m coverage run --source=pizza_order --data-file=.coverage.code -m pytest
CodeCoverageTest_pizza_order.py -q
..............                                                           [100%]
14 passed in 0.07s
Exception ignored in: <function Console.__del__ at 0x000001FE904D04A0>
Traceback (most recent call last):
  File "C:\Users\karib\AppData\Local\Programs\Python\Python311\Lib\site-
packages\pyreadline3\console\console.py", line 256, in __del__
OSError: [WinError -1073741816] Windows Error 0xc0000008
Exit code: 0

> python -m coverage report --data-file=.coverage.code -m --fail-under=90
Name             Stmts   Miss  Cover   Missing
------------------------------------------------
pizza_order.py      34      0   100%
------------------------------------------------
TOTAL               34      0   100%
Exit code: 0

> python -m coverage json --data-file=.coverage.code -o reports/code_coverage.json
Wrote JSON report to reports/code_coverage.json
Exit code: 0

Required minimum: 90% | PASS
```

The pyreadline3 message is a local console cleanup warning after pytest completes; the process returned zero, all assertions passed, and coverage produced its data and report. It is retained in the log for transparency.

Native screenshot to add

Capture the terminal showing the pytest result and the final code coverage table. Save screenshots/code_coverage.png and insert it on this page before submission.

4. Branch coverage evidence - actual command output

The following log comes from an independent --branch run of BranchCoverageTest_pizza_order.py. The application has 24 branch outcomes; all are covered and none is partial or missing.

```
LAB 6 | PES1UG24CS082 | BRANCH COVERAGE
Target: pizza_order.py

> python -m coverage run --branch --source=pizza_order --data-file=.coverage.branch -m pytest
BranchCoverageTest_pizza_order.py -q
.............................                                            [100%]
31 passed in 0.15s
Exception ignored in: <function Console.__del__ at 0x0000016B66A004A0>
Traceback (most recent call last):
  File "C:\Users\karib\AppData\Local\Programs\Python\Python311\Lib\site-
packages\pyreadline3\console\console.py", line 256, in __del__
OSError: [WinError -1073741816] Windows Error 0xc0000008
Exit code: 0

> python -m coverage report --data-file=.coverage.branch -m --fail-under=90
Name             Stmts   Miss Branch BrPart  Cover   Missing
------------------------------------------------------------
pizza_order.py      34      0     24      0   100%
------------------------------------------------------------
TOTAL               34      0     24      0   100%
Exit code: 0

> python -m coverage json --data-file=.coverage.branch -o reports/branch_coverage.json
Wrote JSON report to reports/branch_coverage.json
Exit code: 0

Branch coverage: 24/24 = 100.00%
Required minimum: 90% | PASS
```

Native screenshot to add

Capture the terminal showing the test result, branch coverage table, and the separate 24/24 = 100.00% branch result. Save screenshots/branch_coverage.png and insert it on this page before submission.

5. CodeCoverageTest_pizza_order.py

Separate statement-coverage suite: 14 parametrized cases.

```
"""Statement coverage: execute every executable line in the assigned module."""
import pytest
from pizza_order import pizza_price, order_discount, delivery_status


@pytest.mark.parametrize("size,toppings,cheese,expected", [
    ("medium", 0, False, 350),
    ("small", 3, False, 320),
    ("large", 4, True, 750),
])
def test_pizza_prices(size, toppings, cheese, expected):
    assert pizza_price(size, toppings, cheese) == expected


@pytest.mark.parametrize("size,toppings,cheese,message", [
    ("extra-large", 0, False, "unknown size"),
    ("medium", -1, False, "invalid toppings"),
    ("small", 0, True, "cheese burst not available for small"),
])
def test_invalid_pizza_orders(size, toppings, cheese, message):
    with pytest.raises(ValueError, match=f"^{message}$"):
        pizza_price(size, toppings, cheese)


@pytest.mark.parametrize("total,day,student,expected", [
    (299, "tuesday", True, 0),
    (600, "tuesday", True, 350),
    (600, "saturday", False, 0),
    (600, "monday", False, 60),
])
def test_discount_statements(total, day, student, expected):
    assert order_discount(total, day, student) == expected


@pytest.mark.parametrize("minutes,distance,expected", [
    (20, 11, "not deliverable"),
    (30, 10, "on time"),
    (45, 5, "late"),
    (46, 5, "free pizza"),
])
def test_delivery_returns(minutes, distance, expected):
    assert delivery_status(minutes, distance) == expected
```

6. BranchCoverageTest_pizza_order.py

Separate branch-coverage suite: 31 parametrized cases.

```
"""Decision coverage with threshold pairs and independent expected values."""
import pytest
from pizza_order import pizza_price, order_discount, delivery_status


@pytest.mark.parametrize("size,toppings,cheese,expected", [
    ("small", 0, False, 200),
    ("medium", 0, False, 350),
    ("large", 0, False, 500),
    ("medium", 3, False, 470),  # Last full-price topping.
    ("medium", 4, False, 500),  # First reduced-price topping.
    ("large", 6, False, 710),
    ("medium", 0, True, 450),
    ("large", 4, True, 750),
])
def test_valid_price_branches(size, toppings, cheese, expected):
    assert pizza_price(size, toppings, cheese) == expected


@pytest.mark.parametrize("size,toppings,cheese,message", [
    ("invalid", 0, False, "unknown size"),
    ("medium", -1, False, "invalid toppings"),
    ("small", 0, True, "cheese burst not available for small"),
])
def test_validation_branches(size, toppings, cheese, message):
    with pytest.raises(ValueError, match=f"^{message}$"):
        pizza_price(size, toppings, cheese)


@pytest.mark.parametrize("total,day,student,expected", [
    (0, "monday", False, 0),
    (299, "tuesday", True, 0),
    (300, "tuesday", False, 150),
    (300, "tuesday", True, 200),
    (301, "tuesday", False, 150),  # Integer division floors.
    (300, "saturday", False, 0),
    (300, "saturday", True, 50),
    (300, "sunday", False, 0),
    (300, "sunday", True, 50),
    (300, "monday", False, 30),
    (300, "monday", True, 80),
    (309, "friday", False, 30),
    (310, "friday", False, 31),
])
def test_discount_decisions(total, day, student, expected):
    assert order_discount(total, day, student) == expected


@pytest.mark.parametrize("minutes,distance,expected", [
    (0, 0, "on time"),
    (30, 10, "on time"),
    (31, 10, "late"),
    (45, 10, "late"),
    (46, 10, "free pizza"),
    (20, 10.01, "not deliverable"),
    (46, 11, "not deliverable"),  # Distance decision takes priority.
])
def test_delivery_decisions(minutes, distance, expected):
    assert delivery_status(minutes, distance) == expected
```

7. Reproduction and deliverable checklist

Environment: Python 3.11.9, pytest 9.1.1, coverage.py 7.17.0. Run the following from the Lab6 directory after installing requirements-dev.txt. Separate coverage data files prevent the two measurements from overwriting one another.

```
python -m pip install -r requirements-dev.txt
python -m coverage run --source=pizza_order --data-file=.coverage.code -m pytest
CodeCoverageTest_pizza_order.py -q
python -m coverage report --data-file=.coverage.code -m --fail-under=90
python -m coverage json --data-file=.coverage.code -o reports/code_coverage.json
python -m coverage run --branch --source=pizza_order --data-file=.coverage.branch -m pytest
BranchCoverageTest_pizza_order.py -q
python -m coverage report --data-file=.coverage.branch -m --fail-under=90
python -m coverage json --data-file=.coverage.branch -o reports/branch_coverage.json
python check_branch_coverage.py
```

Branch-only verification helper

```
"""Check the branch-only percentage rather than coverage.py's combined Cover."""
import json

with open("reports/branch_coverage.json", encoding="utf-8") as report:
    totals = json.load(report)["totals"]
covered = totals["covered_branches"]
total = totals["num_branches"]
percentage = 100 * covered / total if total else 100.0
print(f"Branch coverage: {covered}/{total} = {percentage:.2f}%")
if percentage < 90:
    raise SystemExit("Branch coverage is below the required 90%")
```

Deliverable	Status
Separate Python test files	Included; complete source printed in this report
Minimum code coverage of 90%	Verified: 100%
Minimum branch coverage of 90%	Verified: 100%
Raw logs and JSON coverage evidence	Included in reports/
Native terminal screenshots	Outstanding; capture and insert before submission
Separate commits	Prepared separately for code and branch coverage
Google Form	Not submitted; upload the completed PDF after adding screenshots

Instructor form: https://forms.gle/7KzWpGWLqh35M9dj8